



# Alexey Mazurenko

AI ENGINEER / LLM ENGINEER

## CONTACTS

- PHONE  
+7 (977) 270-09-30
- EMAIL  
lyakoway@gmail.com
- FORMAT  
office / hybrid / remote
- WEBSITE  
lyakoway.vercel.app/contacts

## ABOUT ME

- AI engineer** with 7+ years of commercial software development experience and a strong software engineering foundation. Main focus — LLM applications, RAG systems, AI agents and workflow automation with AI.
- Building full-cycle AI products** — from problem research and AI architecture design to development, quality evaluation, integration and production delivery.
- Working with real LLM use cases:** corporate knowledge search, natural-language data analysis, Text-to-SQL, multi-step AI agents, automation of repetitive processes.
- Focused on the quality and reliability of AI systems**, not just answer generation: evaluation sets, golden sets, Recall@K, LLM-as-a-Judge, regression testing, context and token-budget control.
- Combining AI and classic engineering** — implementing backend and frontend on my own, integrating LLMs and external services, working with databases and infrastructure.
- Love turning AI prototypes into working products**, where not only model capabilities matter, but also speed, reproducibility, error resilience and end-user convenience.
- Constantly exploring new AI approaches and tools**, testing hypotheses in practice and adopting what truly improves the product.

## PROFILE

**AI / LLM Engineer** with 7+ years of commercial software development experience. Specialized in building production-ready AI solutions: RAG systems, AI agents and LLM applications with Python. Recent experience — MTC Web Services (MWS AI), the AI division of the MTS ecosystem, one of Russia's largest telecom operators (B2C products).

Building **full-cycle AI systems** — from data processing and retrieval to generation, validation and answer quality evaluation. Implementing **RAG and hybrid search (BM25 + Vector Search, RRF)**, **embeddings**, **Agent Loop (ReAct)**, **Tool Calling**, **Text-to-SQL** and **multi-agent orchestration**.

Focused on the **quality and reliability of LLM systems**: golden sets, Recall@K, LLM-as-a-Judge, regression testing, error handling and self-correction, context and token-budget optimization. Working with cloud and local LLMs via API, including **GLM**, **OpenAI**, **Anthropic** and **Ollama**.

Strong **software engineering foundation** and end-to-end AI product development experience: Python / FastAPI, PostgreSQL / ClickHouse, React / Next.js, Docker / Kubernetes / CI/CD. Able to independently take **an AI prototype all the way to a production solution**.

More about my projects and experience — on my website: [lyakoway.vercel.app](https://lyakoway.vercel.app)

## EXPERIENCE

### AI Engineer

MTC Web Services (MWS AI)

Apr 2024 — present · Moscow

### PROJECTS

#### RAG Chat — AI assistant with Retrieval-Augmented Generation

An AI assistant for working with documents — uploads PDF, Word and Excel files, answers questions about their content and provides links to the source pages and document fragments.

- RAG pipeline:** document indexing → chunking (tiktoken) → embeddings (fastembed) → hybrid BM25 + vector retrieval (RRF) → LLM answer generation → source citation.
- Three modes:** RAG Chat → AI Agent → Vector Search — switching within one application.
- Architecture:** Python / FastAPI → ChromaDB → fastembed → LLM API (GLM / OpenAI / Anthropic) / Ollama → SSE → React / TypeScript; SQLAlchemy — conversation history.
- Quality:** Recall@1 87% on a 47-scenario golden set (evaluation script), LLM-as-judge 5.0/5.

#### AI Data Pilot — multi-agent analytics platform

Automates the path from a user's question to a ready analytical result.

- Oleg — AI analyst:** turns natural-language questions into SQL, runs multi-step data analysis, detects trends and deviations, and produces tables, charts and analytical conclusions.
- Ksyusha — RAG agent:** answers questions over internal technical documentation and uploaded files (PDF, Word, Excel), grounded in the retrieved sources.
- Architecture:** Python / FastAPI → SQLAlchemy → Agent Loop (ReAct) → Tool Calling → Text-to-SQL → SQL guard → PostgreSQL / ClickHouse → analytics layer → SSE → React / TypeScript; Ksyusha's RAG core — hybrid retrieval BM25 + vector embeddings (fastembed) → LLM → source citation.
- Quality:** 174 automated tests + golden-set LLM evaluation (Execution / Result Accuracy, Self-Correction Rate): agent loop, self-correction, SQL Guard, analytics, routers, data sources.

### CORE TASKS

- Designed and built AI agents for business scenarios — workflows, function / tool calling, orchestration, error handling and failure recovery.
- Created AI agents for content generation and development-process automation.
- Developed and iteratively improved prompt scenarios, testing them for accuracy, stability and edge cases.
- Built evaluation sets for regression checks of answer quality and for tracking changes in models and prompts.
- Optimized LLM context handling — compression, prioritization and token-budget management.
- Researched and adopted multi-agent orchestration approaches, keeping up with current industry practices.
- Helped build the AI platform end-to-end — backend (Python, FastAPI), frontend (React / Next.js), infrastructure (Docker, Kubernetes, CI/CD).
- Wrote scripts and helper utilities in Python / Bash, maintained API documentation and worked with open-source projects.
- Explored new approaches, tested hypotheses and shipped best practices to production.

### KEY RESULTS

#### RAG Chat

- The product is finished and used across the company.
- Cuts information lookup from minutes to seconds — the user asks a question in their own words and gets an answer with a link to the specific fragment of the source document.
- Reduces repetitive support requests — customers get answers from the technical documentation on their own, without involving a specialist.
- Speeds up work with internal documents — employees quickly find the right clauses in regulations, instructions and HR policies without manually digging through folders.
- Speeds up legal and financial document review — the system finds the right terms, amounts and figures in contracts and reports in seconds.

SKILLS

Languages

Python TypeScript / JavaScript

LLM & AI Agents

Multi-Provider LLM API (GLM / OpenAI / Anthropic / Ollama)

AI Agents Agent Loop (ReAct)

Tool Calling Text-to-SQL

Multi-Agent Orchestration

Prompt Engineering

Context / Token Optimization

Latency & Cost Tuning (TTFT, \$ per query)

RAG & Search

RAG Hybrid Search (BM25 + Vector)

RRF Cross-Encoder Reranking

Chunking (tiktoken)

Cross-lingual Retrieval (RU / EN)

Embeddings fastembed ChromaDB

Evaluation & Quality

Golden Sets Recall@K / MRR

LLM-as-a-Judge

Grounding / Anti-Hallucination

Observability & Monitoring (GA4 / Metrika / feedback)

Prompt Evaluation Regression Testing

Edge Cases

AI Data & Backend

FastAPI SQLAlchemy PostgreSQL

ClickHouse SQLite Redis

SQL Guard (read-only, timeouts) SSE

Node + Express

Frontend

React + Next Redux Toolkit Svelte

Styled-Components Webpack Jest

Vite

Infrastructure

Docker Kubernetes CI/CD Git

EDUCATION

Master's degree — Applied Mathematics  
MGSU — Institute of Fundamental Education,  
2014

KEY RESULTS

AI Data Pilot

- The product is finished and used across the company.
- Cuts reporting preparation from 2 hours to 2 minutes — the user asks a question in natural language or launches a saved scenario with one click and gets ready data, a chart and a conclusion.
- Eliminates manual SQL work — the agent translates the user's request into SQL, executes it and produces the analysis result on its own.
- Reduces data-related errors — the agent validates execution results, detects SQL failures, rewrites the query itself and re-runs it.
- Automates the path from question to analytical conclusion — the system computes metrics, detects trends and deviations, highlights key changes and explains them to the user.
- Turns recurring requests into ready-made scenarios — an analyst saves a frequently used query and re-runs it without manual SQL or analysis setup.
- Brings heterogeneous data under one interface — corporate databases (PostgreSQL, ClickHouse) and uploaded Excel files are available to the agent simultaneously, with the SQL dialect adapted automatically.

STACK

RAG Chat Python FastAPI SQLAlchemy RAG AI Agents LLM API ChromaDB fastembed

Ollama SSE React TypeScript Vite

AI Data Pilot Python FastAPI SQLAlchemy Text-to-SQL Agent Loop (ReAct) Tool Calling RAG

BM25 + Vector Search fastembed PostgreSQL ClickHouse SSE React 19

TypeScript pytest

PROCESSES

Scrum Jira Confluence

DEMO

- RAG Chat — <https://lyakoway.vercel.app/portfolio/rag-chat>
- AI Data Pilot — <https://lyakoway.vercel.app/portfolio/ai-data-pilot>

EXPERIENCE

Senior Frontend Developer

Feb 2019 — Apr 2024 · Moscow

MTC Web Services

PROJECTS

MTS Profile

A module for storing and visualizing customer data with access management across ecosystem products.

Ecosystem widgets

An embeddable navigation and personalization module for b2c/b2b products.

CORE TASKS

- Building new features and evolving existing products.
- Requirements analysis, technical solution design and integration architecture scenarios.
- Frontend component development and reusable UI solutions with Storybook.
- Refactoring and code reviews, improving code quality and maintainability.
- Unit and integration testing, analysis and resolution of production incidents on 3rd-line support.
- Technical documentation, mentoring and helping onboard new team members.

KEY RESULTS

- Delivered a profile ownership transfer model and a linked-accounts management model.
- Automated user data verification via Gosuslugi with biometrics.
- Delivered the full access management and authorization cycle — access recovery, sign-in and an authentication-method change history.
- Integrated bank card payments and ecosystem widgets into the company's digital products.

STACK

React Next.js TypeScript Redux Toolkit Svelte Styled-Components Webpack Jest Node.js

Express

PROCESSES

Scrum Jira Confluence

DEMO

- MTS Profile — [id.mts.ru](https://id.mts.ru)